Open Brain — Architecture Reference

Internal fleet reference. Written 2026-07-11 by reading the repo (`~/ppp2/open-brain`) end to end — schema, all three edge functions, migrations. The deployed code is exactly what is committed here, so this reflects the live system. Version numbers are pinned at the June 2026 v2 cutover and may have advanced since; the design has not.

Audience: Christopher + the AI fleet (Claude Code across machines, claude.ai, Hermes, Le Chat). Contains live operational detail (project ref, env var names, deploy commands). Not for external sharing.

---

1. What Open Brain is

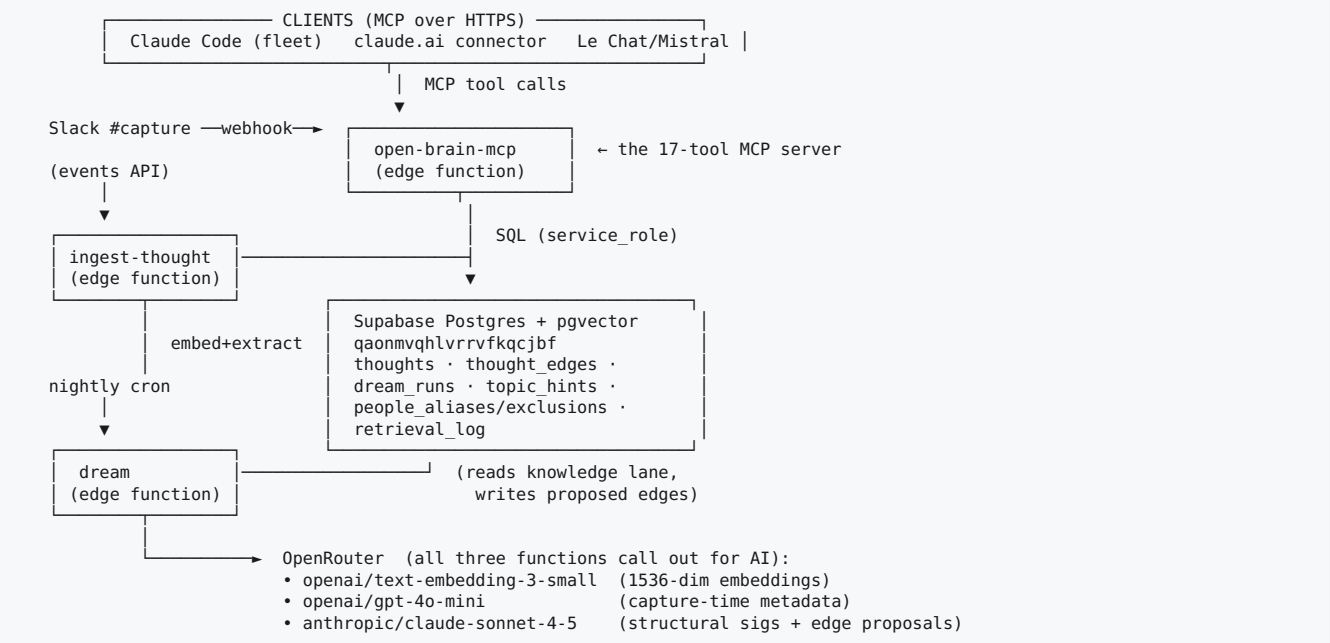
Open Brain is a **Supabase-hosted MCP memory system** — *a two-lane knowledge-graph journal*. AI clients capture semantic "thoughts" and retrieve them later by meaning; a nightly job proposes non-obvious cross-domain connections between the atomic ones, which a human confirms or rejects. It is the shared long-term memory layer across the fleet: what one agent learns, another can recall.

It exists because an LLM with only a context window re-derives everything from scratch every session. Open Brain is the persistence that lets the fleet *compound* knowledge instead — durable state across sessions, machines, and clients.

The honest scope, learned from three months of v1 usage: it works extremely well as a **structured project journal with semantic search and cross-session state transfer**. The "second brain" graph layer is a deliberate, human-curated experiment layered on top (Section 8), not the load-bearing value.

---

2. System topology



Three edge functions, one database, external AI via OpenRouter. Everything server-side runs as Deno edge functions with the Supabase `service_role` key; Row-Level Security is enabled on all tables with a single "service_role full access" policy (defense in depth — the anon/PostgREST path has no access).

Live deployment: Supabase project `qaonmvqhlvrrvfkqcjbf`. At the v2 cutover: `open-brain-mcp` v12, `ingest-thought` v13, `dream` v3, all migrations applied.

---

3. Data model

The single source of truth is `supabase/migrations/`; the live DB is built from these (earliest schema predates the repo and is recorded in `docs/older-docs/SCHEMA_BASELINE_20260610.md`).

`thoughts` — the core table

Column	Type	Notes
`id`	uuid PK	
`content`	text	the raw thought
`embedding`	vector(1536)	pgvector; `text-embedding-3-small`
`metadata`	jsonb	see below
`structural_sig`	jsonb	abstract pattern signature (dream job fills this, knowledge lane only)
`lane`	text	`journal` (default) or `knowledge`; CHECK-constrained
`retrieval_count`	int	read instrumentation; default 0
`last_retrieved_at`	timestamptz	read instrumentation
`created_at` / `updated_at`	timestamptz	

`metadata` jsonb carries: `type` (`observation` / `task` / `idea` / `reference` / `person_note`), `project`, `topics[]`, `people[]`, `action_items[]`, `dates_mentioned[]`, `status` (`active` / `completed` / `superseded` / `archived`), `status_history[]`, `source` (`mcp` / `slack` / `direct`), and for distilled insights `source_session_id`.

`thought_edges` — the insight graph

Column	Type	Notes
`id`	uuid PK	
`source_id` / `target_id`	uuid	the connected pair
`label`	text	`STRUCTURAL_ANALOG` / `SOLUTION_TRANSFER` / `CONTRADICTS` / `PATTERN_INSTANCE`
`weight`	float	proposer confidence (0–1; ≥0.7 to be stored)
`insight`	text	one sentence: the non-obvious connection
`rationale`	text	falsifiable "X because Y" clause (required)
`created_by`	text	`dream`
`confirmed`	bool nullable	`null` = unreviewed, `true` = confirmed, `false` = rejected
`created_at`	timestamptz	

Unique index `uq_thought_edges_pair` on `(least(source,target), greatest(source,target))` — a node pair is proposed **at most once, ever**. Rejected pairs stay in the table (`confirmed=false`) and can never be re-proposed. This makes "never nag me twice" structural, not behavioral.

Supporting tables

- **`dream_runs`** — one row per nightly run: timestamps, `thoughts_processed`, `edges_proposed`, `status`, `error`, `notes`.
- **`topic_hints`** — controlled topic vocabulary: `topic`, `description`, `category`, `active`. Injected into the capture-time extraction prompt so tags converge instead of sprawling (~14 canonical hints after the June consolidation; 50 old ones deactivated but kept).
- **`people_aliases`** — `alias → canonical_name`, collapses name variants.
- **`people_exclusions`** — names hard-stripped from `people[]` (Christopher is never tagged; he's the implicit author of everything).
- **`retrieval_log`** — read instrumentation: `ts`, `tool`, `query`, `result_ids[]`, `client_hint`.
- Legacy/parked: `grendel_thoughts` (empty), `thoughts_backup_20260323`.

Key RPC: `match_thoughts`

The vector-search function behind retrieval. Score = blended similarity + recency, then a lifecycle demotion:

```
score = ( (1 - recency_weight) * cosine_similarity
        +      recency_weight  * recency_decay )
        × (status ∈ {completed, superseded, archived} ? 0.8 : 1.0)
```

Defaults: `match_threshold 0.3`, `recency_weight 0.15`, `max_age_days 365`. The ×0.8 demotion (not hiding) is the v2 fix for stale tasks outranking live ones — **history stays findable, it just stops outranking the present.** A second RPC, `increment_retrieval(ids[])`, bumps read counters in one round-trip.

---

4. The two lanes (the central design decision)

Every thought has a `lane`. This split is the heart of v2.

	`journal` (default)	`knowledge`
Holds	session summaries, tasks, observations, artifacts — working state	atomic insights: one claim, one topic, ≤60 words
Lifecycle	managed: active → completed / superseded / archived	mostly permanent
Graph material?	**No** — the dream job never reads it	**Yes** — the *only* thing the dream job reads
On retrieval	truncated to 80 words + `get_thought` pointer	returned in full (already short)
Atomicity	verbose is fine	**enforced at the API** — capture rejects >70 words or multi-topic

Why split: connecting two 400-word session summaries produces mush; connecting "X works because Y" to "Z fails for the same structural reason" produces an insight. Atomicity is what makes graph edges meaningful, so it is enforced at write time, not left to convention. `capture_thought(lane='knowledge')` runs two gates: a word-count check (≤~70) and an LLM `single_topic` judgment; failing either returns a "split this" error.

> **Fleet note.** Navigational *pointer* thoughts (e.g. "doc X is about Y, fetch via get_artifacts") are deliberately captured on the **journal** lane — they are signposts, not insights, and the knowledge-lane guard correctly rejects them. Reserve the knowledge lane for genuine one-claim insights.

---

5. Capture paths & the ingestion pipeline

Two ways in, same enrichment:

1. **MCP** — `capture_thought` on the `open-brain-mcp` server (the fleet's primary path).
2. **Slack / webhook** — the `ingest-thought` function watches a capture channel (`SLACK_CAPTURE_CHANNEL`); any message becomes a thought and gets a threaded confirmation reply. Also accepts a direct `POST {content}` test mode that bypasses Slack filtering.

Enrichment pipeline (both paths), run in parallel where possible:

```
content ──►  getEmbedding()          → 1536-dim vector (text-embedding-3-small)
        └──► fetchTopicHints()        → active controlled vocabulary
             └► extractMetadata()     → gpt-4o-mini returns {type, topics, people,
                                          action_items, dates_mentioned}
                                          (told to reuse topic hints before inventing)
   then:
     • status defaults to "active"
     • project auto-detected (detectProject: keyword → grendel / open-brain /
       notes-world / geopolitical-sim / … / general)  — or explicit on MCP capture
     • people normalized: alias-mapped, exclusions removed, self-tags dropped
     • INSERT into thoughts
```

Self-tag filtering is emphatic and belt-and-suspenders: the extraction prompt is told never to emit Christopher, and `normalizePeople()` filters him again after (all variants: "christopher rehm", "chris", "c. rehm", ...). Rationale: he was tagged on 141/249 thoughts in v1 — zero information, since he authors all of them.

---

6. Retrieval

Reading is where the value is realized, so v2 instrumented it.

Tools: `search_thoughts` (semantic), `list_thoughts` (filtered browse), `get_thought` (full text of one), `search_with_graph` (semantic entry points + graph expansion), `get_connected` (edges from a given thought).

Ranking: via `match_thoughts` (Section 3) — cosine + recency, lifecycle-demoted. **Truncation:** journal hits return 80 words + a `get_thought` pointer so a session-start pull doesn't dump 3–5k tokens of stale summaries; knowledge hits return whole.

Graph expansion: `search_with_graph` finds semantic entry points, then traverses confirmed `thought_edges` (default depth 1) to surface connected thoughts from *other* domains — the payoff of the whole graph layer. Confirmed edges get traversal priority.

Instrumentation: every read increments `retrieval_count` / `last_retrieved_at` and logs a `retrieval_log` row — **fire-and-forget: a logging failure can never break or slow a search.** `retrieval_stats` turns this into the report v1 couldn't produce: reads by tool, most-retrieved thoughts, never-retrieved share, reads/day. At review time, "X% of thoughts were never read" is data, not a guess.

---

7. The insight graph & the `dream` job

The nightly `dream` function is the graph builder. It runs **only on the knowledge lane** — the single biggest edge-quality fix over v1.

Pipeline:

1. **Budget check.** Hard cap of **10 proposals/week** (`WEEKLY_PROPOSAL_CAP`). If already hit, exit — keeps the human review ritual at ~15 minutes.
2. **Structural signatures.** For knowledge thoughts lacking one, `claude-sonnet-4-5` abstracts the thought to a domain-neutral pattern: `{pattern, problem_type, dynamic, domain, analogy_surface}` (e.g. `feedback_loop_without_brake` , `principal_agent_divergence`). Stored in `structural_sig` .
3. **Candidate pairing.** Group signed thoughts by `pattern` and by `problem_type` ; within each group, form pairs **only across different `domain` s** (cross-domain is the whole point). Pairs sharing a project *and* a topic sort last. Bounded: last 90 days, ≤200 thoughts, ≤100 pairs/run.
4. **Edge judgment.** For each candidate pair, `claude-sonnet-4-5` decides if there's a genuine non-obvious connection. It must return `connected:true` , `weight ≥ 0.7` , and a **falsifiable rationale containing "because"** — "both relate to art" is explicitly rejected. No because-clause → no edge.
5. **Insert** as `confirmed=null` (unreviewed). The unique pair index + a pre-check skip any pair already in the table (including rejected).

Weekly edge review (human, ~15 min): `list_pending_edges` shows each unreviewed proposal with both thoughts in full, the machine's because-rationale, and the date. `confirm_edge(id, true|false)` . **Reject honestly** — confirming weak edges to feel productive corrupts the graph, and a rejected pair is gone for good.

---

8. Design rationale, success metric & kill criteria

The graph layer is a deliberate, **falsifiable** experiment, not an article of faith (this discipline is itself characteristic of how the system is run).

- **Primary success metric:** ≥1 confirmed, genuinely novel cross-domain edge per month.
- **Honest cost:** the graph only pays off with human-in-the-loop curation (~15 min/week + 1–3 atomic insights at session close). No automation substitutes for edge confirmation; unconfirmed machine edges are noise.
- **Kill criteria:** if after **two full months** of followed protocol (≥6 weekly reviews, ≥40 knowledge thoughts) there are **zero** confirmed novel edges, conclude a single-user corpus doesn't support cross-domain discovery — stop the dream job, drop the weekly ritual, **keep** knowledge-lane capture + vector search (atomic insights retrieve well regardless) and **keep** all journal-lane improvements (they stand alone). No zombie features.

---

9. The 17-tool MCP surface

Served by `open-brain-mcp` . Grouped by purpose:

Group	Tools
Capture & lifecycle	`capture_thought` (with `lane`), `update_thought_status`, `distill_session`
Retrieval	`search_thoughts`, `list_thoughts`, `get_thought`, `search_with_graph`, `get_connected`
Graph curation	`list_pending_edges`, `confirm_edge`
Health / metrics	`thought_stats`, `retrieval_stats`
Topics	`add_topic_hint`, `list_topic_hints`
Artifact handoff	`push_artifact`, `get_artifacts`, `complete_artifact`

`distill_session(summary_id, insights[])` is the thin wrapper that writes 1–3 atomic insights to the knowledge lane, each linked to its source journal summary — so the protocol step is one explicit call and the linkage is never forgotten.

External MCP clients (e.g. Le Chat / Mistral) can connect; some get a **read/write tool split** (read-only subset) — see `docs/LECHAT_CONNECTOR.md`.

---

10. Human protocols (the system fails without them)

Encoded in the Open Brain block of `~/.claude/CLAUDE.md`, so Claude Code follows them automatically.

Session close (every working session): 1. **Update statuses** — `update_thought_status` on anything finished/superseded this session. *A memory that is confidently wrong is worse than none.* 2. **Capture the journal summary** — one `capture_thought(lane='journal')`: decisions, blockers, progress, next steps. 3. **Distill** — `distill_session` with 1–3 atomic insights: *what was learned*, not *what was done*. Skip if nothing was learned; padding corrupts the graph.

Weekly (~15 min): edge review (Section 7). **Monthly** (~10 min): `retrieval_stats` + count confirmed-novel edges against the ≥1/month target; trigger the Section 8 decision if failing.

---

11. Fleet usage layer — context docs on top of artifacts

The artifact tools (`push_artifact` / `get_artifacts` / `complete_artifact`) were designed for claude.ai → Claude Code handoff, but the fleet now also uses them as a **shared, reusable reference-doc store**, organized in **two tiers by retrieval cost** (built July 2026):

Project	Role	How it's pulled
`context-core`	the small always-on set — behavioral charter, values, how-to-think, programming rules	`get_artifacts("context-core", status="all")` **in full at session start**
`context-docs`	the larger on-demand library — art, business, biography, templates, etc.	a *specific* doc only when the task touches its domain; **never bulk-loaded**

Why two tiers: `get_artifacts("context-docs", status="all")` grew past the tool's output-token limit (~87k chars) — "load everything at session start" became impossible. Splitting an always-on core (kept small on purpose) from an on-demand library fixes it and keeps startup cheap. `priority` (`high` / `normal` / `low`) tags the tier intent.

Discovery: each doc has a matching **journal-lane pointer thought** (`Context doc pointer: "<slug>" …`) so `search_thoughts` surfaces it. Pointers use the journal lane by design (Section 4 note).

Guardrails baked into the convention: - **Never upload credentials** — Open Brain is a shared brain with no per-doc access control. - **Sensitive docs are pointer-only** — the content stays local; the artifact is a stub instructing any AI to *stop and ask Chris*. The content never touching the cloud is the only real "ask first" gate.

Session-start wiring: `~/.claude/CLAUDE.md` instructs Claude Code to pull `context-core` at session start and fetch domain docs on demand. Claude.ai carries the same instruction in its profile; Hermes calls it in its startup. The procedure is captured as the **`add-context-doc`** skill (fleet-synced under `~/.claude/skills/` → `~/ai-workzone/AI-general-docs/claude-skills/`).

---

12. Operations

- **Repo (single source of truth):** `~/ppp2/open-brain` — all three functions, migrations, scripts, docs. Deployed code == committed code.

- **Deploy:** `supabase functions deploy <name> --no-verify-jwt` (`open-brain-mcp` / `ingest-thought` / `dream`).
- **Migrations:** idempotent (`if not exists` everywhere), one file per work item, separately revertible; the live DB is built from them.
- **Secrets (env, never committed):** `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `OPENROUTER_API_KEY`, `SLACK_BOT_TOKEN`, `SLACK_CAPTURE_CHANNEL`, `DREAM_SECRET`.
- **AI models (via OpenRouter):** `openai/text-embedding-3-small` (embeddings, 1536-dim — fixed, no model changes), `openai/gpt-4o-mini` (capture metadata), `anthropic/claude-sonnet-4-5` (structural signatures + edge proposals).
- **Auth posture:** MCP deployed `--no-verify-jwt` (access gated by the connector's MCP key); `dream` gated by `DREAM_SECRET` (header or query param); `ingest-thought` filters to one Slack channel and ignores bot/subtype events. RLS on every table; only `service_role` has access.
- **Anti-fragility (hard requirement across the codebase):** each handler isolates its failure domain — per-tool try/catch, structured JSON error logging, no top-level crash. New columns are nullable-or-defaulted so old code paths survive mid-deploy. Instrumentation is fire-and-forget.

---

13. Known limitations & open questions

- **Graph value is unproven** by design — the Section 8 metric/kill-criteria exist precisely because a single-user corpus may not support cross-domain edge discovery. Watch `dream_runs` and the weekly review.
- **Retrieval value is still being measured** — `retrieval_stats` was added to answer "is this memory worth anything?"; the 6-month read data is the arbiter.
- **`get_artifacts(status="all")` on a large project exceeds the output-token limit** — mitigated by the context-core/context-docs tiering (Section 11); spot-check inventories by grepping the saved result file for `^Title:` / `^Status:` rather than reading the whole payload.
- **Metadata extraction is LLM-judged** — `single_topic` and topic tagging are model calls, so occasionally imperfect; the controlled vocabulary and self-tag filters constrain the damage.
- **No per-document access control** — everything in the shared brain is readable by any connected client; hence the credential-exclusion and pointer-only rules for anything sensitive.

Appendix — file map

```
~/ppp2/open-brain/
├── README.md                                doc index + deploy one-liners
├── docs/
│   ├── ARCHITECTURE.md                       ← this file (full reference)
│   ├── OPEN_BRAIN_PLUS_PLUS.md               v2 change narrative (why it changed)
│   ├── LECHAT_CONNECTOR.md                   external MCP client setup
│   └── older-docs/                           v2 spec, pre-v2 schema baseline, topic proposal
├── supabase/
│   ├── functions/
│   │   ├── open-brain-mcp/index.ts           the 17-tool MCP server
│   │   ├── ingest-thought/index.ts           Slack/webhook capture + enrichment
│   │   └── dream/index.ts                    nightly edge proposer
│   └── migrations/                           schema (source of truth)
└── scripts/                                  one-off SQL (topic remap, self-tag strip)
```